

LV13 포인터의 활용(배열, 함수)

<https://youtu.be/ORXMceJRJ4w?feature=shared>

```
void Increase(int pArr[7])
{
    for (size_t i = 0; i < 7; i++)
    {
        pArr[i]++;
    }
}

int main()
{
    int arr[7] = { 123, 43, 2, 3, 5, 7 };
    Increase(arr);

    return 0;
}
```

C++ 포인터와 배열 심화 학습

포인터 변수의 활용 (return 값 두개이상 받기)

함수에서 여러 값을 반환하는 방법

일반적으로 함수는 return문을 통해 하나의 값만 반환할 수 있습니다. 하지만 포인터를 활용하면 여러 개의 값을 동시에 반환하는 효과를 얻을 수 있습니다.

기본적인 return문 (값 하나만 반환)

```
#include <iostream>
using namespace std;

int Increase(int a)
{
    return a + 1;
}
```

포인터를 이용한 다중 값 변경

```
#include <iostream>
using namespace std;
```

```

void Increase(int a, int b)
{
    a += 1; // 포인터가 가리키는 원본 변수의 값을 1 증가
    b += 1; // 포인터가 가리키는 원본 변수의 값을 1 증가
}

void Increase(int* a, int* b)
{
    *a += 1; // 포인터가 가리키는 원본 변수의 값을 1 증가
    *b += 1; // 포인터가 가리키는 원본 변수의 값을 1 증가
}

void Insert(int* a, int* b)
{
    *a = 100; // 원본 변수에 100 대입
    *b = 100; // 원본 변수에 100 대입
}

int main()
{
    int num1 = 10;
    int num2 = 20;

    // 값을 1씩 증가시키는 함수 호출
    num1 = Increase(num1); // 일반 함수: 반환값을 다시 대입해야 함
    num2 = Increase(num2);

    // 포인터를 이용한 함수 호출: 원본 변수가 직접 변경됨
    Increase(&num1, &num2);

    return 0;
}

```

핵심 개념:

- **Call by Value:** 값을 복사하여 전달, 원본 변수는 변경되지 않음
- **Call by Reference:** 주소를 전달하여 원본 변수를 직접 변경
- 포인터를 사용하면 함수에서 여러 변수를 동시에 수정할 수 있음

배열을 인자로 함수에 넘기기

배열과 포인터의 관계

배열의 이름은 배열의 첫 번째 원소의 주소를 나타냅니다. 따라서 배열을 함수에 전달할 때는 실제로 포인터가 전달됩니다.

중요한 특징:

- 배열도 결국 다수의 int 변수의 나열이기 때문에 모든 변수의 주소를 보낼 필요가 없고 시작 주소 하나만 보내면 됩니다
- 즉, int arr[]의 주소만 보내면 됩니다. 결국 int변수 주소를 보내는 것과 같습니다

1차원 배열을 함수로 전달

```

#include <iostream>
using namespace std;

void bbq(int* pArr /*int pArr[10]*/)
{
    *(pArr) = 100;    // pArr[0] = 100과 동일
    *(pArr + 1) = 100; // pArr[1] = 100과 동일
    *(pArr + 2) = 100; // pArr[2] = 100과 동일
    *(pArr + 3) = 100;
    *(pArr + 4) = 100;
    *(pArr + 5) = 100;
    *(pArr + 6) = 100;
    *(pArr + 7) = 100;
    *(pArr + 8) = 100;
    *(pArr + 9) = 100; // pArr[9] = 100과 동일

    // 배열 표기법도 동일하게 사용 가능
    pArr[0] = 200;
    pArr[1] = 200;
    pArr[2] = 200;
    pArr[3] = 200;
    pArr[4] = 200;
    pArr[5] = 200;
    pArr[6] = 200;
    pArr[7] = 200;
    pArr[8] = 200;
    pArr[9] = 200;
}

int main()
{
    int vect[10] = {};

    // 배열의 이름이 배열의 첫번째 원소의 주소와 같다.
    // 그렇기에 배열을 함수로 전달할 때는 배열의 시작주소이다.
    bbq(&vect[0] / vect);

    return 0;
}

```

포인터 산술 연산 설명:

- `(pArr + 1)` : pArr이 가리키는 주소에서 1칸(4바이트) 뒤의 위치
- `pArr + 1` 이라고 해서 주소값에 1을 더하는 것이 아니라, 해당 자료형의 데이터 크기만큼 이동
- int형은 4바이트이므로 실제로는 주소값에 4가 더해짐

배열 전달의 장점과 특징

포인터 변수를 활용하는 제일 큰 이유는 배열 전체를 복사해서 전달할 것은 너무 큰 일이고 비효율적이어서 배열의 크기만 보내면 8바이트(64bit) 만 복사하면 됩니다.

다만 `*(pArr + 1)` 이런 방식으로 사용하면 상당히 코드를 작성하는데 불편하다. 그래서 문법상 포인터변수는 배열처럼 바로 사용이 가능하다.

다만 주의해야 할 점이 전달된 배열의 크기를 넘어가지 않게 주의해서 사용해야 한다.

배열에 이름은 해당 배열의 시작 주소이다. 그래서 이름을 전달했을때 넘겨주는시에 복사가 되는것이 아니라 원본이 그대로 넘어간다. (주의해야한다!)

간단한 배열 전체 증가 함수

```
#include <iostream>
using namespace std;

void Increase(int* pArr /*int pArr[7]*/)
{
    for (int i = 0; i < 7; i++)
    {
        pArr[i]++;
    }
}

int main()
{
    int arr[7] = { 123, 42, 2, 66, 7, 9, 10 };

    Increase(arr); // 배열의 모든 원소가 1씩 증가

    return 0;
}
```

디버거에서 확인할 수 있는 내용:

- 그래서 조사식에서 배열에 이름을 넣으면 값에 주소가 보여진다.

arr	0x00000023c9ffb28 {123, 43, 2, 3, 5, 7, 0} int[7]
[0]	123 int
[1]	43 int
[2]	2 int
[3]	3 int
[4]	5 int
[5]	7 int
[6]	0 int

- 배열의 주소: `0x00000023c9ffb28 {123, 43, 2, 3, 5, 7, 0} int[7]`
- 각 원소의 값과 타입이 표시됨

2차원 배열을 포인터 변수로 전달해야 할 때의 문법이 약간 다르다

2차원 배열의 메모리 구조 이해

2차원 배열은 메모리상에서는 1차원으로 연속적으로 저장됩니다. 하지만 함수로 전달할 때는 특별한 문법이 필요합니다.

```
#include <iostream>
using namespace std;
```

```

// 2차원 배열을 매개변수로 받는 방법 1
void kfc2D(int (*pArr2D)[3])/(*int pArr2D[3][3]*)/
{
    pArr2D[0][0] = 100;
    pArr2D[0][1] = 100;
    pArr2D[0][2] = 100;

    pArr2D[1][0] = 100;
    pArr2D[1][1] = 100;
    pArr2D[1][2] = 100;

    pArr2D[2][0] = 100;
    pArr2D[2][1] = 100;
    pArr2D[2][2] = 100;
}

// 2차원 배열을 매개변수로 받는 방법 2
void kfc(int* pArr /*int pArr[3]*/)
{
    pArr[0] = 100;
    pArr[1] = 100;
    pArr[2] = 100;
}

int main()
{
    int arr[3][3] = {};
    //int** pArr2D = arr; // 틀리다.
    int(*pArr2D)[3] = arr; // 이렇게 해야한다.

    // 본질은 1차원배열을 여러번 나누어서 2차원배열을 사용한다.
    // 그러한 가정 은 이루는 가능성 ( 하나의 함수는 하나의 일만 하도록 만들어야 한다. )
    kfc2D(arr);

    // 각 행을 개별적으로 전달
    kfc(arr[0]); // 첫 번째 행
    kfc(arr[1]); // 두 번째 행
    kfc(arr[2]); // 세 번째 행

    return 0;
}

```

2차원 배열 전달 방법:

1. `int (*pArr2D)[3]` : 포인터 to 배열 문법 사용
2. `int pArr2D[][3]` : 첫 번째 차원은 생략 가능, 두 번째 차원은 반드시 명시
3. 각 행을 개별적으로 1차원 배열로 전달

메모리 구조 설명:

- 2차원 배열 `arr[3][3]` 는 메모리상에서 9개의 int가 연속으로 배치

- `arr[0]`, `arr[1]`, `arr[2]` 는 각각 3개 원소를 가진 1차원 배열의 시작 주소

최대값과 최소값 찾기 (인덱스까지 함께)

값과 위치를 동시에 찾는 알고리즘

단순히 최대값, 최소값만 찾는 것이 아니라 해당 값이 배열의 몇 번째 위치에 있는지도 함께 찾는 방법입니다.

```
#include <iostream>
using namespace std;

int main()
{
    int vect[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int max = INT_MIN; // 가능한 가장 작은 정수값으로 초기화
    int min = INT_MAX; // 가능한 가장 큰 정수값으로 초기화

    int maxIdx = -1; // 최대값의 인덱스 (초기값은 유효하지 않은 -1)
    int minIdx = -1; // 최소값의 인덱스 (초기값은 유효하지 않은 -1)

    for (int i = 0; i < 10; i++)
    {
        if (max < vect[i])
        {
            max = vect[i]; // 최대값 갱신
            maxIdx = i;    // 최대값의 위치 저장
        }

        if (min > vect[i])
        {
            min = vect[i]; // 최소값 갱신
            minIdx = i;    // 최소값의 위치 저장
        }
    }

    cout << "최대값: " << max << ", 위치: " << maxIdx << endl;
    cout << "최소값: " << min << ", 위치: " << minIdx << endl;

    return 0;
}
```

알고리즘 특징:

- **INT_MIN, INT_MAX:** `<climits>` 헤더에 정의된 정수의 최소값, 최대값 상수
- **인덱스 초기화:** -1로 초기화하여 유효하지 않은 상태를 나타냄
- **동시 탐색:** 한 번의 반복문으로 최대값과 최소값을 동시에 찾음

2차원 배열에서 최소값과 최대값 위치 기억하기

2차원 배열에서는 행(row)과 열(column) 정보를 모두 저장해야 합니다.

```
#include <iostream>
using namespace std;
```

```

int main()
{
    int vect[3][3] =
    {
        3, 4, 1,
        5, 5, 7,
        8, 3, 2,
    };

    int min = 987654321; // 충분히 큰 값으로 초기화
    int dy = 0;          // 최소값의 행(row) 좌표
    int dx = 0;          // 최소값의 열(column) 좌표

    for (int y = 0; y < 3; y++)
    {
        for (int x = 0; x < 3; x++)
        {
            if (vect[y][x] < min)
            {
                min = vect[y][x]; // 최소값 갱신
                dy = y;           // 최소값의 행 좌표 저장
                dx = x;           // 최소값의 열 좌표 저장
            }
        }
    }

    cout << "최소값: " << min << endl;
    cout << "위치: (" << dy << ", " << dx << ")" << endl;
    cout << "즉, vect[" << dy << "][" << dx << "] = " << min << endl;

    return 0;
}

```

2차원 배열 탐색의 특징:

- **이중 반복문:** 행과 열을 순차적으로 탐색
- **좌표 저장:** (y, x) 형태로 2차원 위치 정보 저장
- **실용적 활용:** 게임에서 맵의 특정 지점 찾기, 이미지 처리에서 특정 픽셀 찾기 등에 활용

성능 최적화 팁:

- 최대값과 최소값을 동시에 찾으려면 조건문을 분리하여 각각 검사
- 배열의 첫 번째 원소로 초기화하면 INT_MIN, INT_MAX보다 안전
- 동일한 값이 여러 개 있을 때는 첫 번째로 발견된 위치가 저장됨

“강의는 많은데, 내 실력은 왜 그대로일까?”

혼자서 공부하다 보면

이런 생각 들지 않으셨나요?

- 강의는 다 듣고도 직접 코드는 못 짜겠고,
- 복습할 땐 어디서부터 다시 시작해야 할지 막막하고,
- 질문하려 해도 물어볼 사람이 없고,
- 유튜브 영상도 정답만 보고 따라 치는 느낌...

그렇다면 지금이 바로

“나만을 위한 코칭”이 필요한 순간입니다.

당신도 할 수 있습니다.

지금 멤버십을 넘어, 코칭에 도전해보세요.

수많은 수강생들이 **암암코딩 코칭**으로 넥슨, 크래프톤, NC 등 입사에 성공했습니다.

 **프리미엄 코칭 안내 바로가기**

 또는 카톡 오픈채팅: **암암코딩 상담방**

지금도 코딩을 ‘따라 치기만’ 하고 계신가요?

이젠 **혼자 설계하고, 스스로 코딩하는 법**을 배워야 할 때입니다.

암암코딩이 옆에서 함께하겠습니다. 

▼ oopy:hide

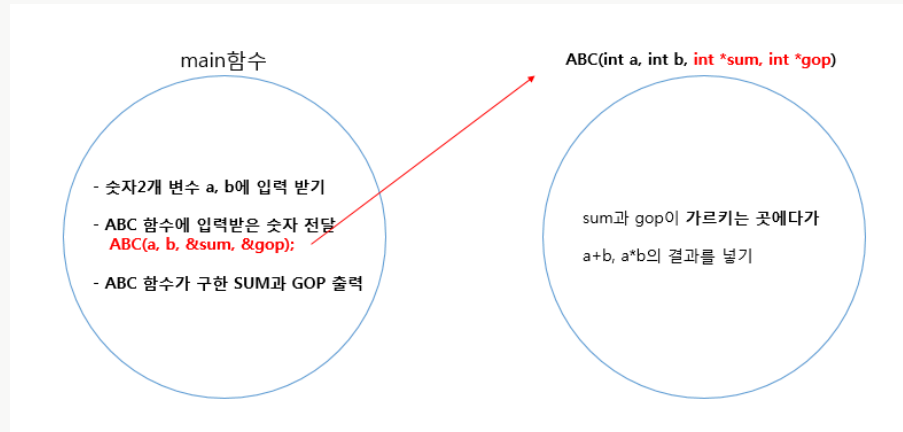
연습문제



문제 1번

main함수에서 숫자2개를 입력받고, ABC 함수에 입력받은 값을 전달 해 주세요

ABC 함수는 전달받은 숫자 2개의 합과 곱을 계산 해 주는 함수입니다



[TIP] 특정 함수에서 숫자 여러개 return 해주기

일반적으로 함수는 한 개의 숫자만 return 할 수 있습니다.

```
void ABC( )
```

```
{
```

```
    return 5;
```

```
}
```

아래와 같이 짜면 에러가 발생하지요

```
void ABC( )
```

```
{
```

```
    return 5, 6; //에러 , return 에 값 두개를 쓸 수 없습니다
```

```
}
```

값을 2개 이상 돌려주는 방법은

1. return 대신 전역변수를 쓴다

2. return 대신 포인터를 쓴다

이렇게 두 가지 방법이 있습니다.

입력 예시

5 6

출력 예시

11 30



문제 2번

다음 배열을 **전역배열**로 하드코딩 해 주세요

A	D	F
Q	W	E
Z	X	C

main 함수에서 찾을 문자를 입력받고, Find함수에 입력받은 문자를 보내주세요. (Find함수 호출)

Find함수에서는 전달 받은 문자가 있는 좌표를 찾아주세요

main함수에서 Find함수가 찾은 좌표를 출력 해 주세요

(전역변수를 사용하지 말고 포인터를 사용해서 문제를 풀어주세요)

입력 예시

F

출력 예시

0,2



문제 3번

한 문장을 main함수에서 입력 받아주세요

strlen이라는 함수를 만들고, 입력받은 문장을 strlen으로 전달 해 주세요 (함수 호출)

strlen함수에서 전달받은 문장의 길이를 구하고 **문장의 길이를 return** 해 주세요

main에서는 문장의 길이를 출력하면 됩니다.

입력 예시

BBQWORLD

출력 예시

8글자



문제 4번

main함수에서 KFC함수를 호출 해 주세요

KFC함수에서는

- 한 문장 (최대 10글자)을 입력받아주세요
- 그 문자에서 대문자 / 소문자가 각각 몇개인지 Counting 해 주세요

main 함수에서는 KFC함수에서 구한 대/소문자 갯수를 각각 출력 해 주세요

(전역배열 / 전역변수를 사용하지 말고, 포인터를 이용 해 주세요)

입력 예시

MINcoding

출력 예시

대문자3개

소문자6개



문제 5번

- 전역변수를 사용하지 않고, 포인터를 활용해서 문제입니다

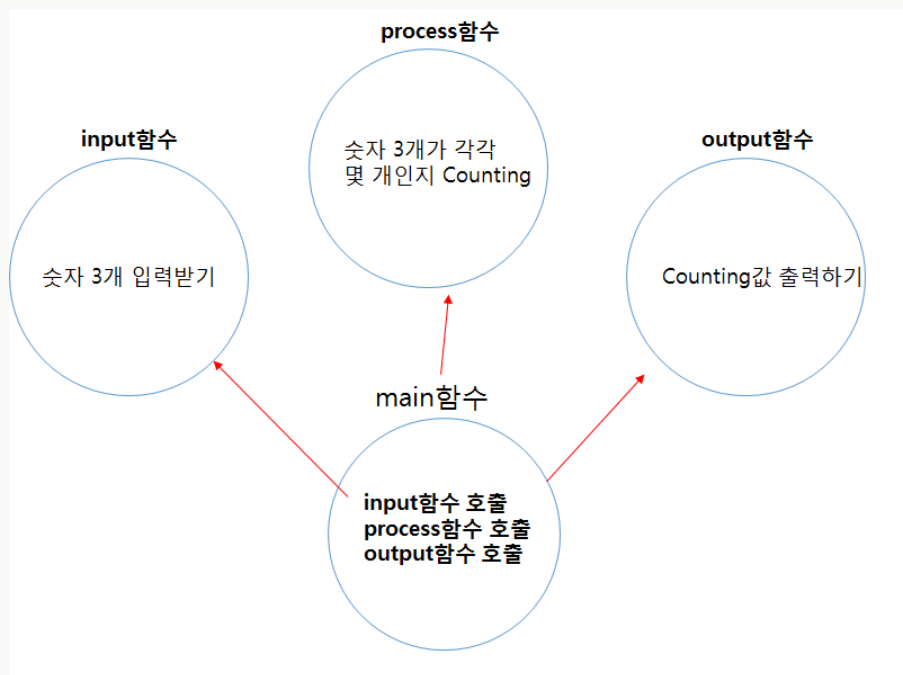
아래의 배열을 전역으로 선언 및 하드코딩 해 주세요

4	5	6	1	3	1
2	1	3	6	3	6

Input함수에서 좋아하는 숫자 3개를 입력받아주세요

Process함수에서는 좋아하는 숫자가 각각 몇개인지 Counting 해 주세요

Output함수에서는 좋아하는 숫자의 갯수를 출력 해 주세요



[TIP] argument

숫자 4개를 받는 함수 ABC가 있다고 한다면,

ABC함수를 호출하기 위해서 이렇게 쓸 수 있습니다

ABC(1, 2, 3, 4);

여기서 보내는 값인 1, 2, 3, 4를 argument(아규먼트)라고 부릅니다

- > ABC함수를 호출하기 위해서는 argument 4개를 써야 합니다

[HINT] 이 문제를 풀기 위해서는

input함수는 argument가 3개 (포인터 3개) 를 써야합니다

ex) main함수에서 input(&a, &b, &c);

process함수는 argument가 6개 (변수3개, 포인터3개) 를 써야합니다

ex) main함수에서 process(a, b, c, &r1, &r2, &r3);

output함수는 argument가 6개를 써야합니다

ex) main함수에서 output(a, b, c, r1, r2, r3);

입력 예시

1 2 3

출력 예시

1=3개

2=1개

3=3개



문제 6번

아래 2차배열을 하드코딩해주세요

이 2차배열은 문자가 들어있기 때문에 int가 아니라 char 2차 배열 입니다.

전부 문자열 처리하여 하드코딩하면 됩니다.

0 ~ 9 사이의 숫자를 입력받고, 그 숫자가 존재하는 해당하는 칸을 찾으세요

그리고 그 칸에 써있는 알파벳을 출력 해 주세요

4	5	7	1	3	2
D	F	Q	W	G	Z

ex) 숫자 3을 입력받으면, 숫자 3의 위치를 먼저 찾아주세요

그리고 그 숫자 밑에있는 알파벳 G를 출력하시면 됩니다

입력 예시

1

출력 예시

W



문제 7번

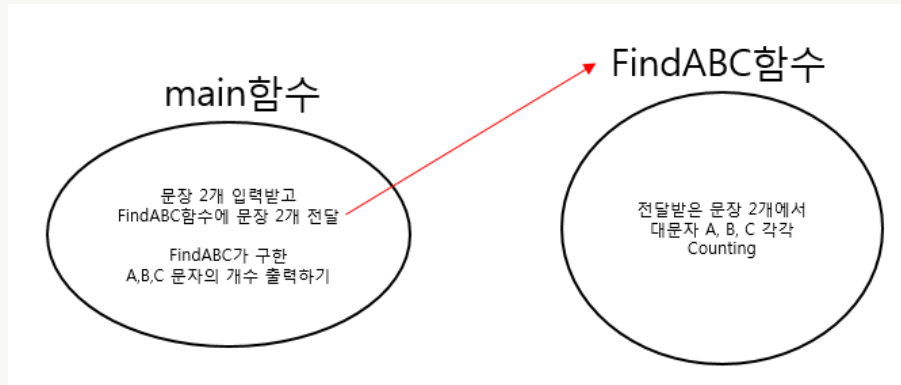
- 전역배열 / 전역변수를 쓰지 않고 푸는 문제입니다

main함수에서 문장 2개를 입력받으세요

그리고 2개의 문장을 FindABC 함수로 전달 해 주세요

FindABC 함수에서는 대문자 A / 대문자 B / 대문자 C를 각각 Counting하는 함수입니다

main함수에서는 FindABC함수를 통해서 구한 A, B, C의 개수를 출력 해 주세요



입력 예시

A_ShowABC

ABCDEFGF

출력 예시

A:3

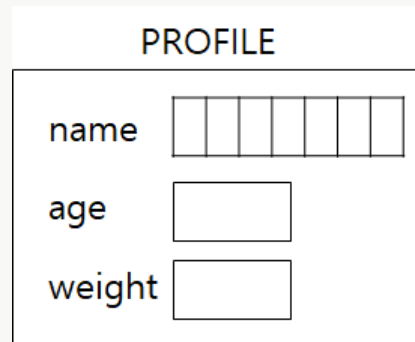
B:2

C:2



문제 8번

아래와 같은 구조체를 먼저 정의 해 주세요



구조체 변수 int1, int2 를 만들어 주세요

이 구조체 변수에 들어갈 내용을 입력 받고

평균을 구해서 출력 해 주세요

name = char 배열

age = int 변수

weight = int 변수

[TIP] 정의하다 라는 뜻

1. 함수 정의

함수를 호출하려면 함수를 미리 만들어 놔야 합니다.

함수를 미리 만들어 두는 것을 "함수를 정의하다" 라고 표현합니다

2. 구조체 정의

구조체 변수를 만드려면 미리 구조체 Type을 만들어 두어야 합니다

구조체 Type을 미리 만들어 두는 것을 "구조체 정의하다" 라고 표현합니다

구조체 변수를 만드는 것은 "선언하다" 라고 부릅니다.

정의하다 vs 선언하다 이 단어를 구분할 줄 알아야 합니다.

"정의하다"라는 용어 알아두세요!

입력 예시

JASON

35

80

TOMS

20

40

출력 예시

JASON & TOMS

평균27세



문제 9번

아래 배열을 전역으로 하드코딩 해 주세요

(전역변수를 쓰지않고 푸는 문제입니다)

D	A	S
Q	W	V
R	T	Y

main함수에서

- 좌표 2쌍(숫자 4개) 입력 받아주세요 (y축, x축)
- Find함수를 호출하는데 입력받은 숫자를 넘겨주세요

Find함수는 좌표 2쌍에 해당하는 값을 찾아주는 함수입니다

main함수에서 좌표에 해당하는 값을 출력 해 주세요

입력 예시

0 2

1 1

출력 예시

S W



문제 10번

아래 표는 나라별 거리를 나타낸 표입니다

A	B	C	D	E	F	G
4	2	5	1	6	7	3

알파벳은 나라를 의미합니다

A 나라에서 E 나라까지 가려면 B, C, D를 거쳐야 합니다

따라서 거리는 $2 + 5 + 1 = 8$ 이 됩니다

민스는 지표를 만나기 위해 얼마나 멀리 떨어져 있는지 알아보려고 합니다

민스가 있는 나라와, 지표가 있는 나라를 입력 받고

두 사람의 거리를 계산해서 출력 해 주세요

ex) 만약 E B를 입력받았다면

E와 B 사이에 D와 C가 있으니까 $1 + 5 = 6$ 이 정답입니다

[힌트] 알파벳을 인덱스로 바꾸는 방법

```
char ch1, ch2;
```

```
cin >> ch1 >> ch2;
```

```
int aIndex = ch1 - 'A';
```

```
int bIndex = ch2 - 'A';
```

입력 예시

E B

출력 예시

6



문제 11번

배열A					
	3	5	1	2	7
배열B					
x	1	2	1	5	9
배열C					
+	7	7	9	5	4
	10	17	10	15	67

1×5 배열 A, B, C 3개를 만들고 숫자 15개를 입력 받으세요.

배열 A와 배열 B의 각각 같은 index값 끼리 곱을 합니다.

그리고 배열 C칸과 같은 index끼리 합을 구한 결과를 출력 하세요.

ex)

입력: => 출력: 10 17 10 15 67

3 5 1 2 7

1 2 1 5 9

7 7 9 5 4

입력 예시

3 5 1 2 7

1 2 1 5 9

7 7 9 5 4

출력 예시

10 17 10 15 67



문제 12번

3	4	1	6
3	5	3	6
5	4	6	0

위 배열을 하드코딩 해 주세요

현재 세번째 Line은 비어져 있습니다.

숫자 4개를 for문으로 입력받고, 세번째 Line을 채워주세요

For문을 돌려서

MAX와 MIN값을 찾아내고, 각 MAX와 MIN값의 좌표를 출력 해 주세요

ex) 9 2 1 1 을 입력하였다면 아래와 같이 배열이 만들어집니다

3	4	1	6
3	5	3	6
9	2	1	1
5	4	6	0

그리고 MAX는 9, MIN값은 0 입니다

가장 빨리 찾아지는 MAX와 MIN값의 좌표를 출력하면 됩니다

입력 예시

9 0 1 1

출력 예시

MAX=9(2,0)

MIN=0(2,1)